```
# remove foreign key by name
remove_foreign_key :accounts, name: :special_fk_name
```

3.7 When Helpers aren't Enough

If the helpers provided by Active Record aren't enough you can use the `execute` method to execute arbitrary SQL:

```
Product.connection.execute('UPDATE `products` SET `price`=`free` WHERE
1')
```

For more details and examples of individual methods, check the API documentation. In particular the documentation for `ActiveRecord::ConnectionAdapters::SchemaStatements` (which provides the methods available in
the `change`, `up` and `down` methods), `ActiveRecord::ConnectionAdapters::TableDefinition` (which provides the methods available on the object yielded by `create_table`)
and `ActiveRecord::ConnectionAdapters::Table` (which provides the methods available on the object yielded by `change_table`).

3.8 Using the `change` Method

The `change` method is the primary way of writing migrations. It works for the majority of cases, where Active Record knows how to reverse the migration automatically. Currently, the `change` method supports only these migration definitions:

- `add_column`
- `add_index`
- `add_reference`
- `add_timestamps`
- `add_foreign_key`
- `create_table`
- `create_join_table`
- `drop_table` (must supply a block)
- `drop_join_table` (must supply a block)
- `remove_timestamps`
- `rename_column`
- `rename_index`
- `remove_reference`
- `rename_table`

`change_table` is also reversible, as long as the block does not call `change`, `change_default` or `remove`. If you're going to need to use any other methods, you should use `reversible` or write
the `up` and `down` methods instead of using the `change` method.

3.9 Using `reversible`

Complex migrations may require processing that Active Record doesn't know how to reverse. You can use `reversible` to specify what to do when running a migration what else to do when reverting it. For example:

```
class ExampleMigration < ActiveRecord::Migration
  def change
    create_table :distributors do |t|
      t.string :zipcode
    end

    reversible do |dir|
      dir.up do
```